Université Abdelmalek Essaâdi
Faculté des Sciences de Tétouan
Département d'Informatique

Filière : Ingénierie Informatique et Développement Logiciel (2IDL) – S5

Année universitaire : 2026- 2027

Cours du Module M353 : Analyse et Conception Orientées Objet en Langage UML

Chapitre 1 : Introduction à l'analyse, COO et UML

Pr. Abderrahim EL MHOUTI

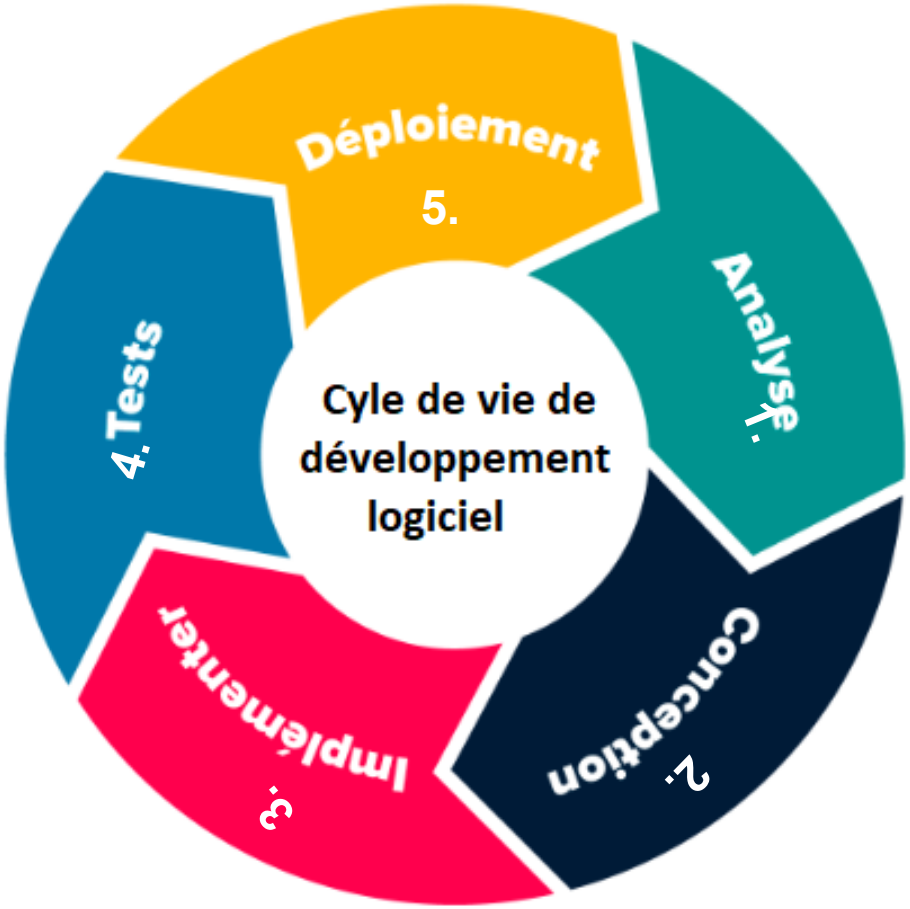
a.elmhouti@uae.ac.ma

Plan du chapitre 1 : Introduction à l'analyse, COO et UML

- **Cycle de vie d'un logiciel**
- **Analyse et Modélisation en ingénierie logicielle :**

  principales méthodes
- **Présentation d'UML**
- **Concepts de base de la modélisation OO avec UML**
- **Vues et diagrammes UML**

Cycle de vie d'un logiciel

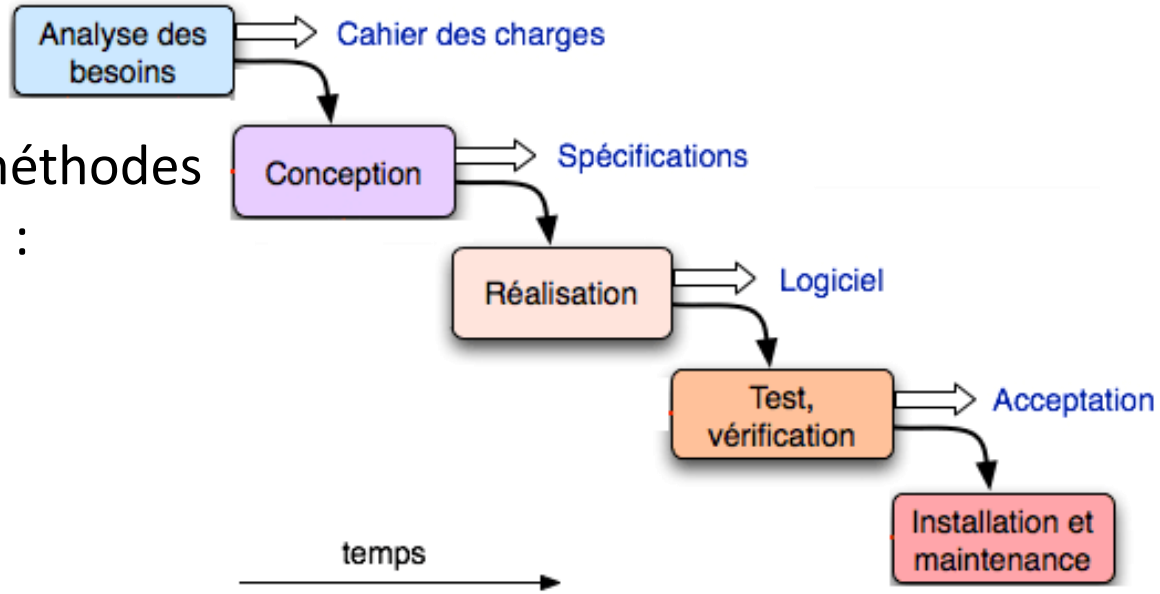


Cycle de vie d'un logiciel
Définition

- Pour obtenir un **logiciel de qualité**, il faut en maîtriser le **processus de développement** :
  - La **vie d'un logiciel** est composée de différentes **étapes**.
  - La succession de ces étapes forme le « **cycle de vie du logiciel** ».
  - Il faut contrôler la succession de ces différentes **étapes**.

- ***Exemples :***

  Exemples de modèles et méthodes de développement logiciel :
  - *Scrum*
  - *Cycle en V*
  - *Cycle en cascade*
  - *Cycle en spirale*
  - *UP (Unified Process), …*

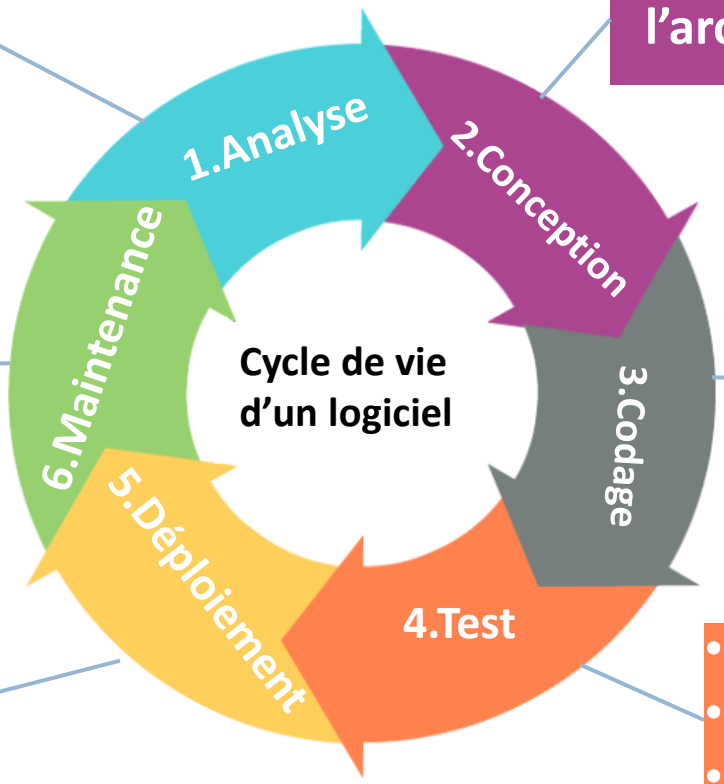


Cycle de vie d'un logiciel

Principales étapes

Spécification des besoins du client

Spécification de la conception et de l'architecture du logiciel



- Assistance aux clients
- Correction des bugs
- Améliorations logicielles

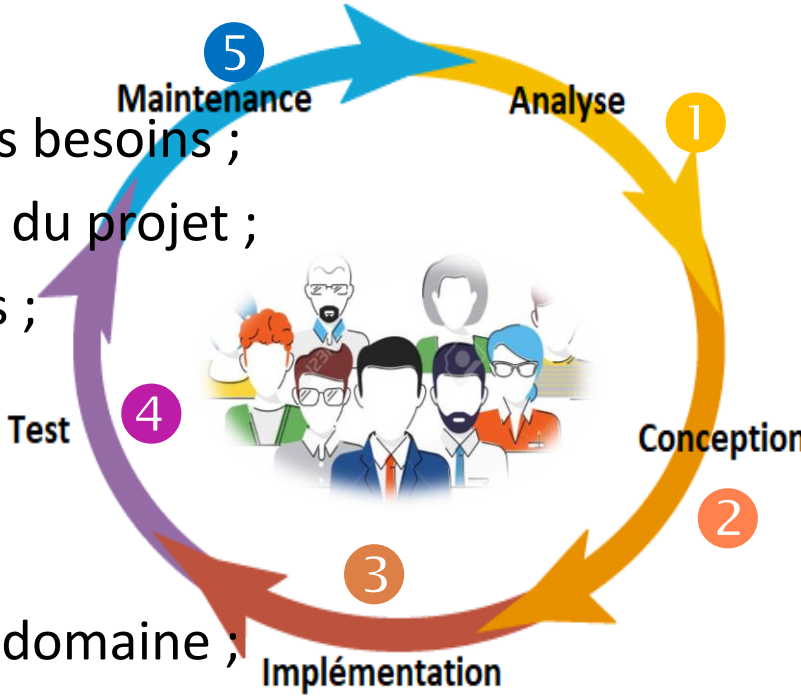
Codage du logiciel

Installation du logiciel dans l'environnement de production

- Tests unitaires
- Tests d'intégration
- Test de performance
- Tests de sécurité

Cycle de vie d'un logiciel
Participants et Rôles

- Le développement d'un **système logiciel** nécessite la collaboration de plusieurs personnes (**participants**) :
  - ***Maître d'ouvrage*** : le **client** qui fournir les besoins ;
  - ***Maître d'œuvre*** : chargé de la **réalisation** du projet ;
  - ***Analyste*** : recueillir et clarifier les besoins ;
  - ***Concepteur :*** modélise le système ;
  - ***Développeur :*** construire le système ;
  - ***Gestionnaire de projet ;***
  - ***Utilisateur :*** fournir les connaissances du domaine ;
  - ***Rédacteur technique ; ….***



- L'ensemble des **tâches** associées à un ***participant*** est appelé « **Rôle** ». Un même ***participant*** peut jouer plusieurs **rôles**.

Cycle de vie d'un logiciel

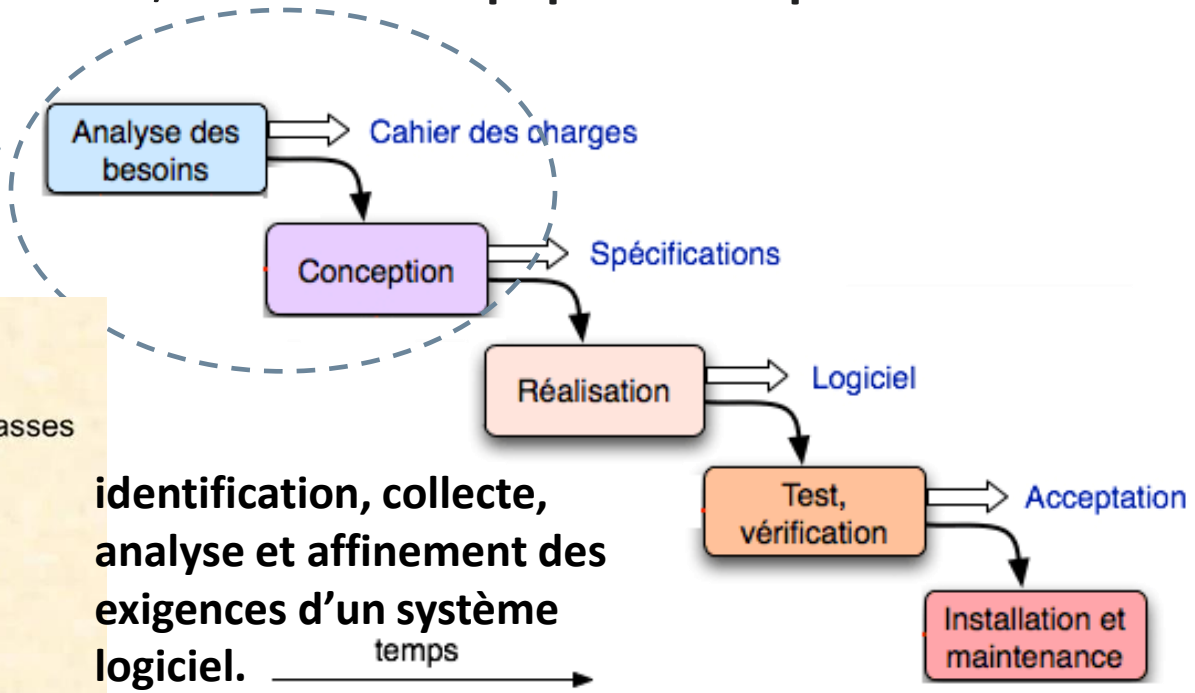
❶ Analyse des besoins

- Le rôle du l'**analyste** est de ***comprendre***, ***formaliser*** et ***valider*** les **besoins** afin de les traduire en spécifications claires pour les ***concepteurs*** et les ***développeurs***.
- Il fait le lien entre les **utilisateurs**/métier et l'**équipe technique**.



Analyste (analyste fonctionnel)

- **Performs many project tasks.**
  - ✓ define business requirements
  - ✓ identify project stakeholders and user classes
  - ✓ elicit requirements
  - ✓ analyze requirements
  - ✓ document requirements in various ways
  - ✓ lead requirements validation
  - ✓ facilitate requirements prioritization
  - ✓ manage requirements

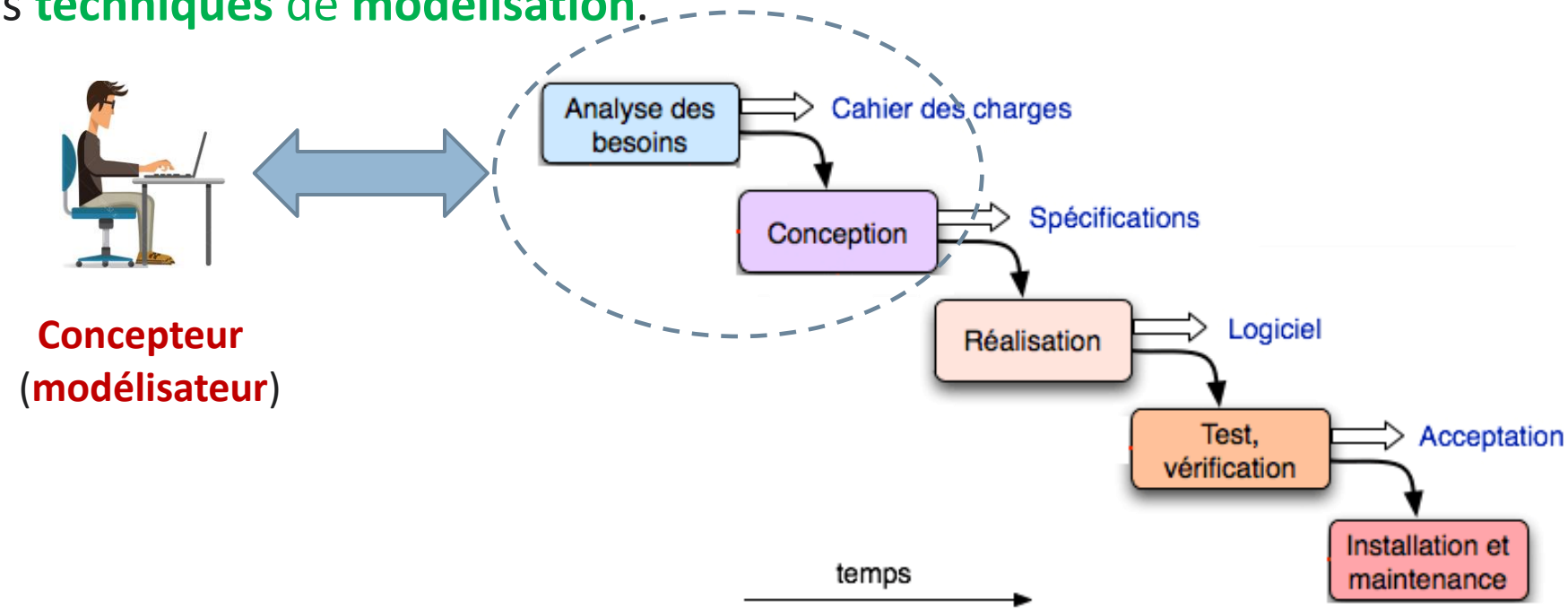


identification, collecte, analyse et affinement des exigences d'un système logiciel.

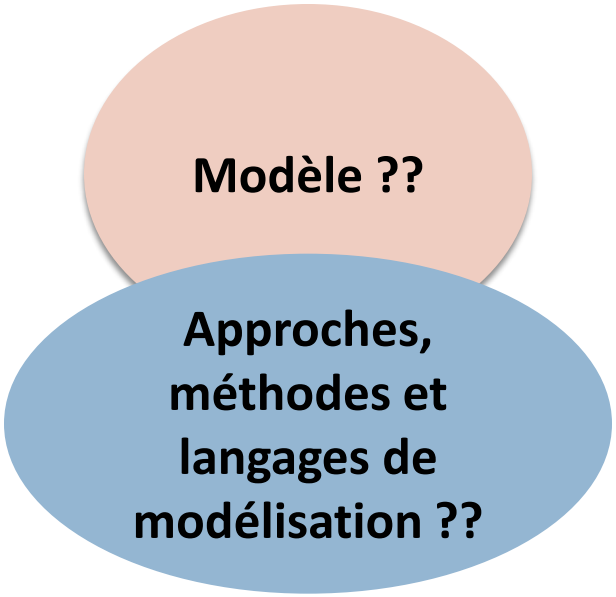
Cycle de vie d'un logiciel

❷ Conception

- Le rôle du **concepteur** (**modélisateur**) est de fournir une **description des fonctionnalités** de l'application sous forme d'un **modèle théorique**.
- Ce **modèle** servira de **plan de construction** de l'application.
- C'est en général dans la phase d'**analyse/conception** que s'appliquent les **techniques** de **modélisation**.



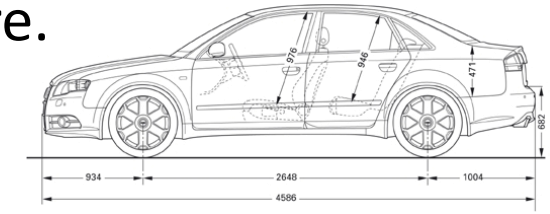
Modélisation en ingénierie logiciel



Modélisation en ingénierie logiciel

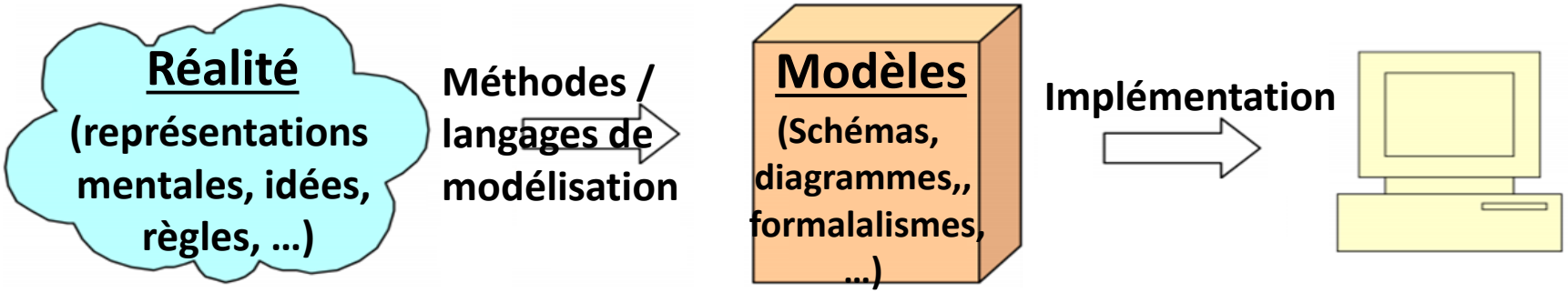
Pourquoi modéliser?

- Généralement, un **système** est de plus en plus **complexe** à comprendre.





- La **construction** d'un **système informatique** de taille importante et par de nombreuses personnes oblige à **modéliser**.



- **Objectif** : **simplifier le système** et par conséquent **faciliter la communication** entre les intervenants.

Modélisation en ingénierie logiciel

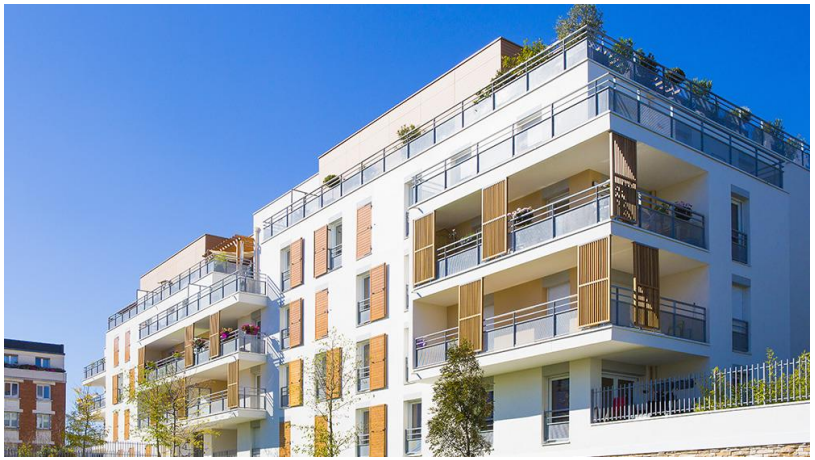
Qu'est ce qu'un modèle?

- Un modèle est une **représentation abstraite et simplifiée** (exclut des détails) d'une **entité** (système, logiciel, bâtiment, etc.) du monde réel en vue de le **décrire**, de l'**expliquer** ou de le **prévoir**.

- **Exemple** : Un modèle (plan) d'une maison :



Représentation abstraite

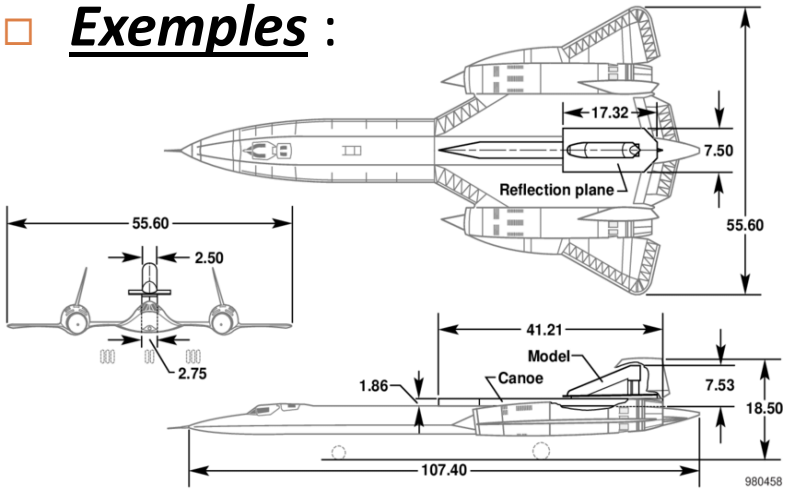


Construction réelle

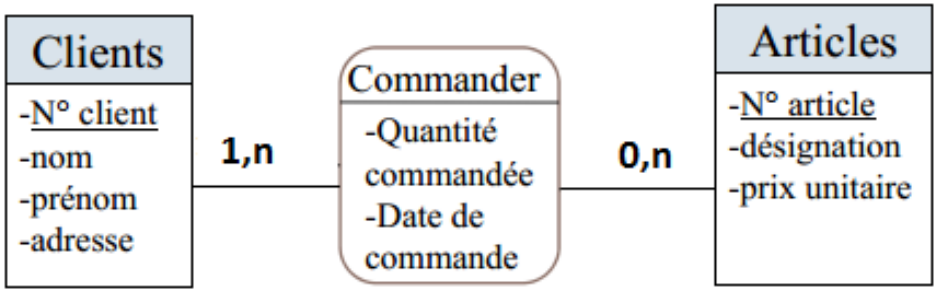
Modélisation en ingénierie logiciel

Comment représenter un modèle?

- Le **modèle** est présenté sous forme de **schéma** ou **diagramme** dans lequel on trouve des **éléments graphiques** et du texte.
- ***Exemples*** :



Modèle conceptuel d'un avion R-71



Modèle conceptuel d'une base de données d'un site e-commerce

- Le **modèle** peut remplacer une longue **description textuelle** par un **schéma** ou **diagramme** plus **facile** et plus **rapide** à interpréter.

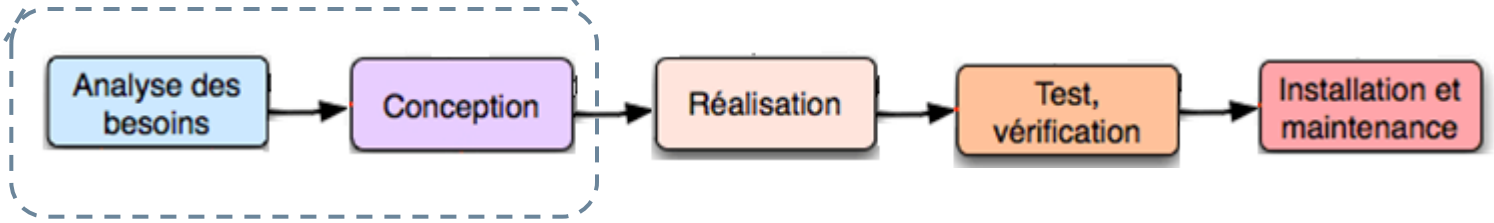
Modélisation en ingénierie logiciel

Qui doit modéliser?

- La **modélisation n'est pas une spécialité** mais plutôt **une aptitude** que peut acquérir un ***ingénieur***, un ***gestionnaire***, un ***analyste,*** ..



- Dans un **projet informatique**, la **modélisation** est souvent faite par le **maitre d'œuvre** en coordination avec le **maître d'ouvrage** pour bien réussir le projet.



- Selon son champ d'intervention, le **modélisateur** (**concepteur**) doit **créer**, **documenter** et **communiquer** son **modèle**.

Modélisation en ingénierie logiciel

Comment modéliser?

- Pour **modéliser** un **logiciel** (système informatique), on utilise des :
  - **Méthodes de modélisation :** procédés permettant de construire un **modèle** aussi correct que possible et aussi efficacement que possible (MERISE, SADT, UP, …).
  - **Langages de modélisation** : une syntaxe commune, graphique, pour modéliser (OMT, OOSE, OOD, **UML**, …).
- Les **méthodes** et **langages** **de modélisation** **obéissent** souvent à des **approches de modélisation** :
  - **approche fonctionnelle** (structurée).
  - **approche orientée objet**.

Modélisation en ingénierie logiciel

Comment modéliser?

Deux principales **approches de modélisation en ingénierie logicielle :**

- **Approche fonctionnelle (structurée) :** (1960 – fin 1970)
  - trouve son origine dans les **langages procéduraux** ;
  - sépare nettement les **données** des **traitements** ;
  - la **fonction** est le moyen d'organisation du **logiciel** ;
  - Exemples : MERISE, SADT, UP, …

- **Approche orientée objet :** (1980 – début 1990)
  - l'**objet** est le moyen d'organisation du **logiciel** ;
  - **Objet** = **données** + **traitements** ;
  - Exemples : *OMT*, **UML**, …

UML

(Unified Modeling Language)

UML (Unified Modeling Language)
Définition

- **UML = Unified Modeling Language = Langage de modélisation unifié.**

- C'est un **langage de modélisation** graphique à base de **diagrammes** utilisé en **conception orientée objet** et en **développement logiciel** .

- **UML** : standard défini par l'**Object Management Group** (OMG).

  - **Standard** : https://www.omg.org/spec/UML/
  - **Site Web** : https://www.uml.org/
  - **Dernière Version : 2.5.1 (décembre 2017)**

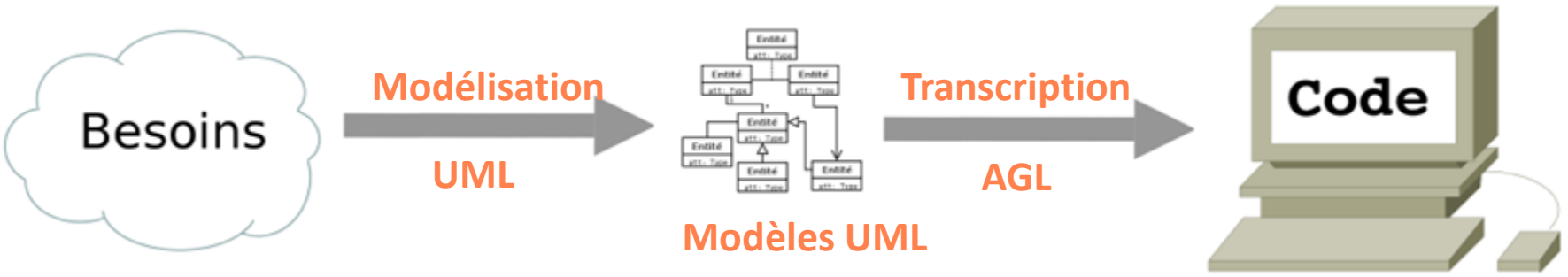


UML (Unified Modeling Language)
Présentation

- Sans **méthodes/langages** de **modélisation**, le **processus de développement** dépendra des personnes, des applications, des cultures, etc.

- ➔ **UML** a été proposé afin de **standardiser** les outils de développement : modèles, notations, diagrammes, …

- UML est un **langage de modélisation** utilisable à la fois par les **humains** (forme graphique) et les **machines** (syntaxe précise).

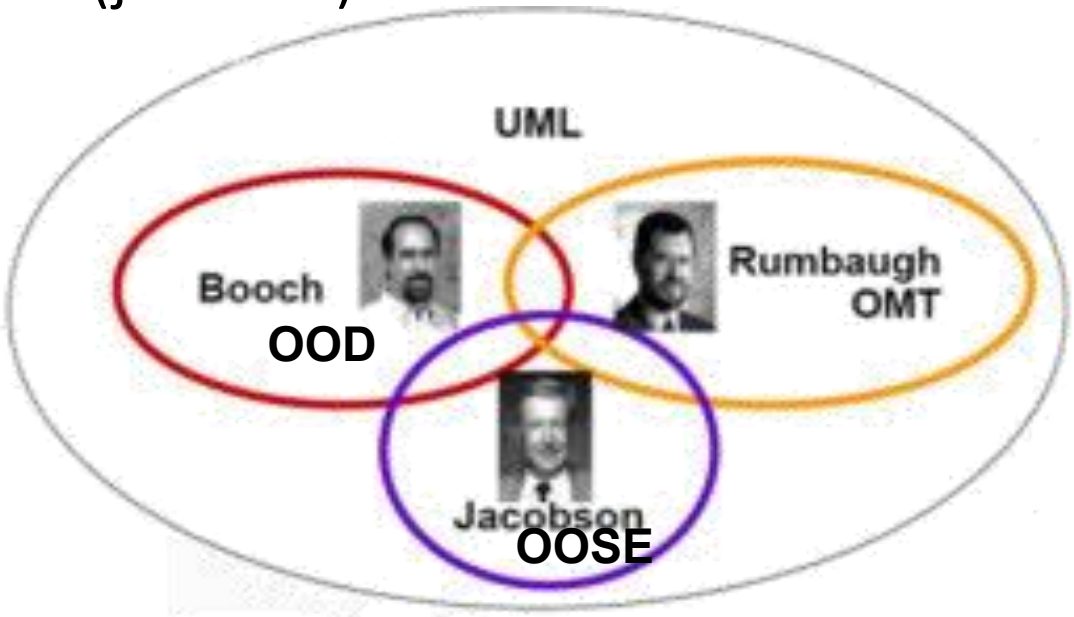


UML (Unified Modeling Language)
Historique

- **Début des années 1990** : **plus de 50** méthodes de **COO** sont apparues, mais seule 3 méthodes ont dominé le marché :
  - **1993** : **OOD** (*Object Oriented Design*) créée par **Grady Booch** : ne couvre pas les étapes de **spécification** et **d'analyse**, elle se concentre sur la phase de **conception** et de **codage**.
  - **1995** : **OOSE** (*Object Oriented Software Engineering*) créée par **Ivar Jacobson** : couvre toutes les étapes du processus de développement des logiciels.
  - **1996** : **OMT** (*Object Modeling Technique*) créée en par **James Rumbaugh** : ne couvre pas l'étape de **spécification**, elle se concentre sur l'étape d'**Analyse** et de **Conception**.

UML (Unified Modeling Language)
Historique

- UML est né (**1997**) de la **fusion** de **3 méthodes** : **OMT**+**OOD**+**OOSE**
  - ➔**UML 1.0** (jan 1997)



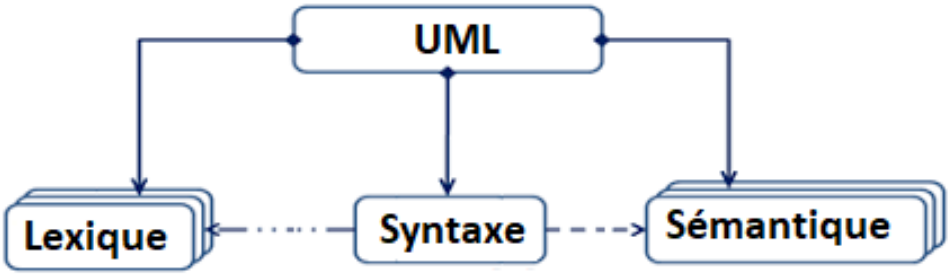
- **Fin 1997 :** l'**OMG** (Object Management Group) retient **UML 1.1** comme **norme** de **modélisation**.
- Dernière version : **2.5.1** (décembre 2017).

UML (Unified Modeling Language)

UML est un langage et non pas une méthode …

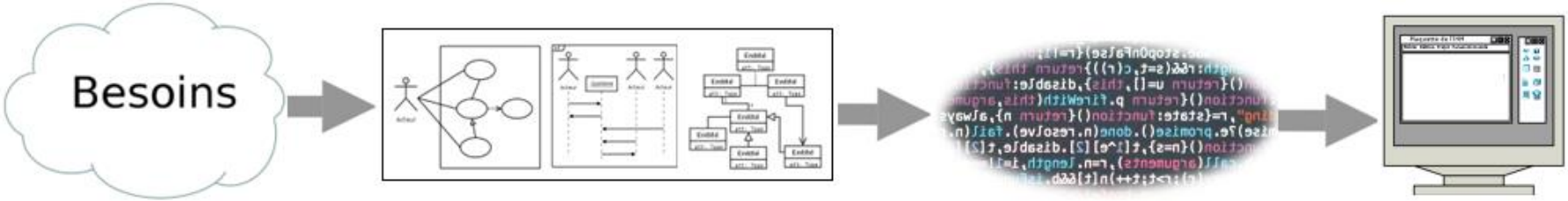
- UML n'est pas une **méthode** (décrivant des étapes de modélisation), mais c'est un **langage graphique OO** qui permet de **représenter** et de **communiquer** les **divers aspects** d'un **système.**

- **Langage** = **lexique** + **syntaxe** + **sémantique**.
  - **lexique** : **mots** du dictionnaire. UML définie une lexique graphique.
  - **syntaxe** = **règles** par lesquelles les éléments du lexique (mots) sont assemblées en expressions (phrases, clauses).
  - **sémantique** = **règles** par lesquelles on donne un **sens** aux expressions syntaxiques.



UML (Unified Modeling Language)
Pourquoi UML?

- UML pour :
  - Obtenir une **modélisation OO** de très **haut niveau** indépendante des **langages** et des **environnements**.
  - Faire **collaborer** des **participants** de tous horizons autour d'un même document de synthèse.
  - Faire des **simulations** avant de construire un système.
  - Exprimer dans un seul **modèle** tous les **aspects du logiciel** : aspect statique, dynamique, juridique, spécification, etc...
  - Documenter le projet informatique.
  - Générer automatiquement la partie logicielle d'un système, …



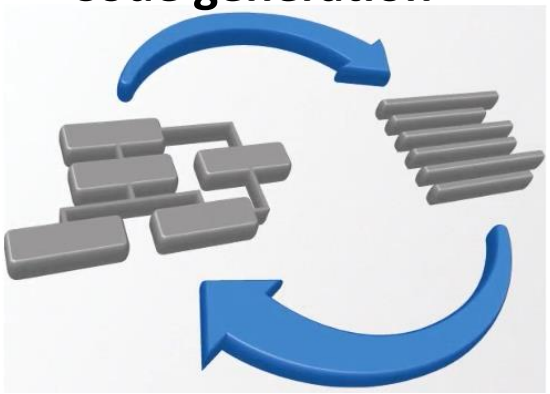
UML (Unified Modeling Language)
Utilisation

- En pratique, la modélisation avec **UML** se fait à l'aide des **Ateliers Génie Logiciel** (AGL).
- En utilisant un **AGL** :
  - toute modification sur les **modèles** vient modifier le **code**.
  - toute modification du **code** est automatiquement répercutée sur les **modèles**.
- ***Exemples*** d'AGL :
  - *Modelio*
  - *Rational Rose*
  - *ArgoUml*
  - *PowerAMC, …*

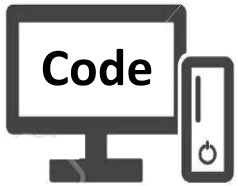


Model

Code generation



Reverse Engeneering

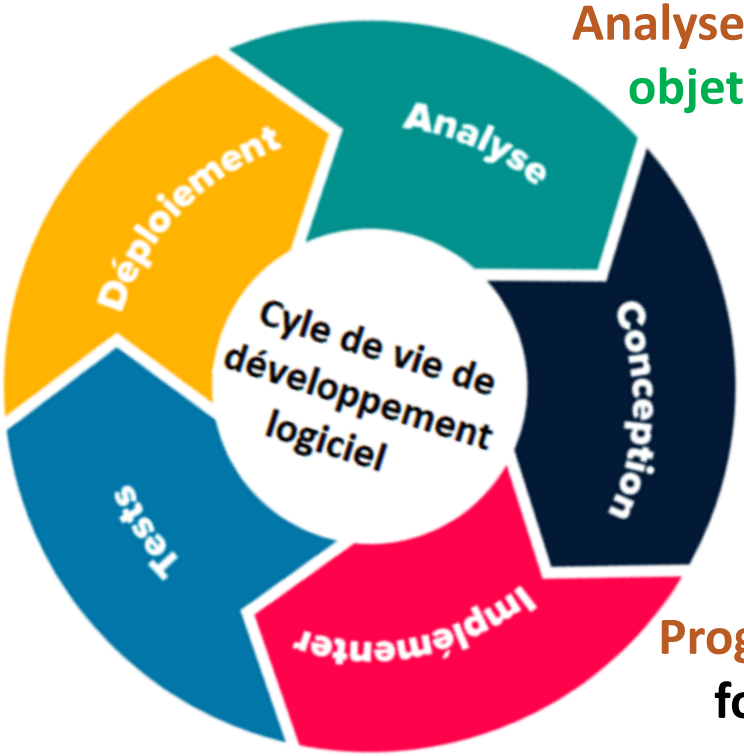


Concepts de la modélisation OO avec UML

Concepts de la modélisation OO avec UML

Ingénierie logicielle d'un point de vue OO

- Du point de vue **OO,** toutes les **phases d'ingénierie** d'un logiciel sont fondées sur le concept d'**objet** :



Analyse OO : examiner les besoins en termes de classes et objets utilisés dans domaine du problème étudié.

Conception OO : utiliser la notation OO pour représenter les modèles logiques et physiques du système.

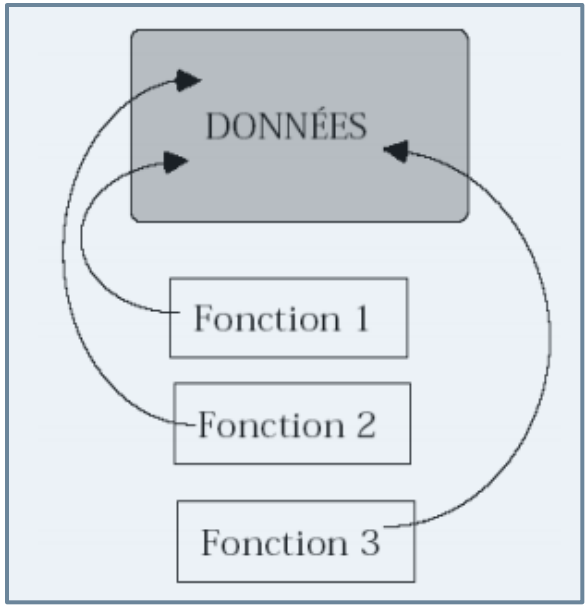
Programmation OO : implémenter l'application sous forme d'une collections coopératives d'objets.

- **➔ Augmenter la productivité et la qualité du développement des logiciels**.

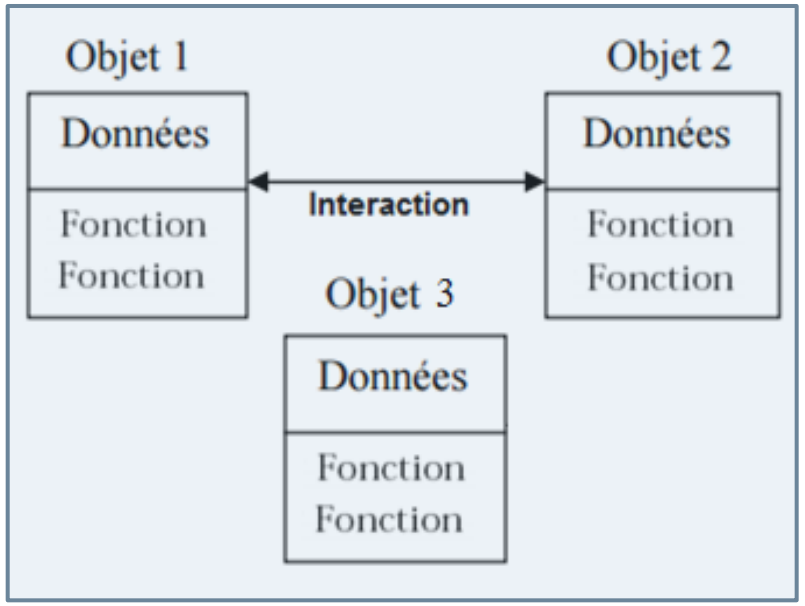
Concepts de la modélisation OO avec UML

Approche fonctionnelle Vs. Approche OO

- **Approche fonctionnelle**: **programme** = ensemble de **fonctions** agissants sur des **données**.
- **Approche OO**: **programme** = ensemble d'**objets**, chacun disposant d'une partie **données** et d'une partie **fonctions** (**méthodes**).



De l'appr. fonctionnelle vers l'appr. OO



Programme écrit avec l'approche fonctionnelle

Programme écrit avec l'approche OO

Concepts de la modélisation OO avec UML

L'objet, un concept principal …

- L'**approche OO** consiste à organiser les **programmes/logiciels** sous formes de collections coopératives d'**objets**.
- Un **objet** représente un **concept** ou une **entité** du **monde réel**.
- L'orientation **objet** est un concept apparu d'abord dans les communautés des **langages de programmation** OO :
  - 1960 : 1er **langage de programmation OO**
  - 1980 : 1ères **méthodes de modélisation OO**
- **Exemples d'objets** :



Objet 1 : Voiture



Objet 2 : Table



Objet 3 : Livre



Objet 4 : Etudiant

Concepts de la modélisation OO avec UML

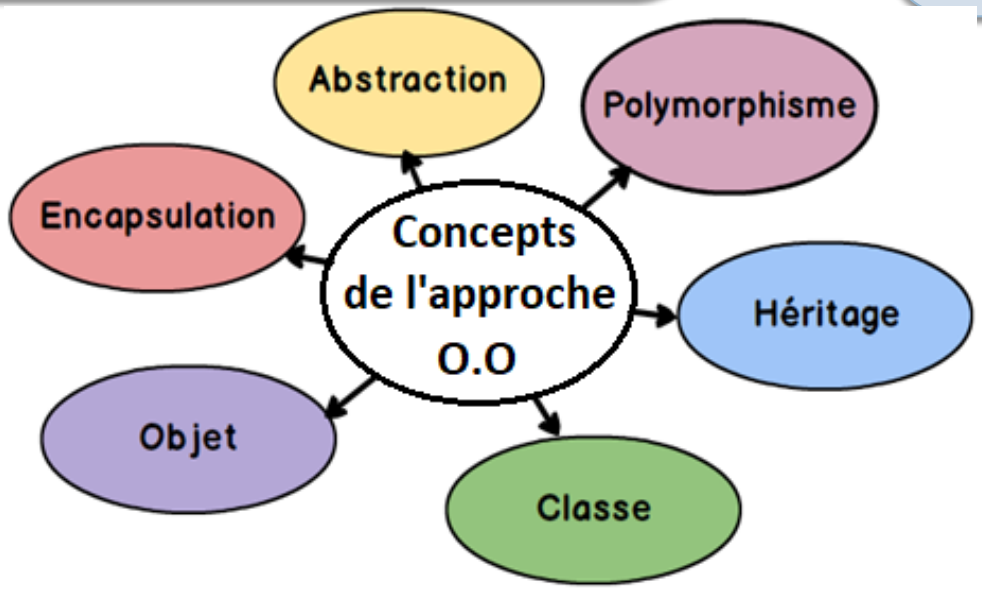
Analyse OO Vs. Analyse fonctionnelle

Analyse fonctionnelle

s'intéresse aux **fonctions** que doit réaliser le système (conception procédurale)

Analyse OO

s'intéresse aux **objets** qui se trouvent dans le domaine d'application.



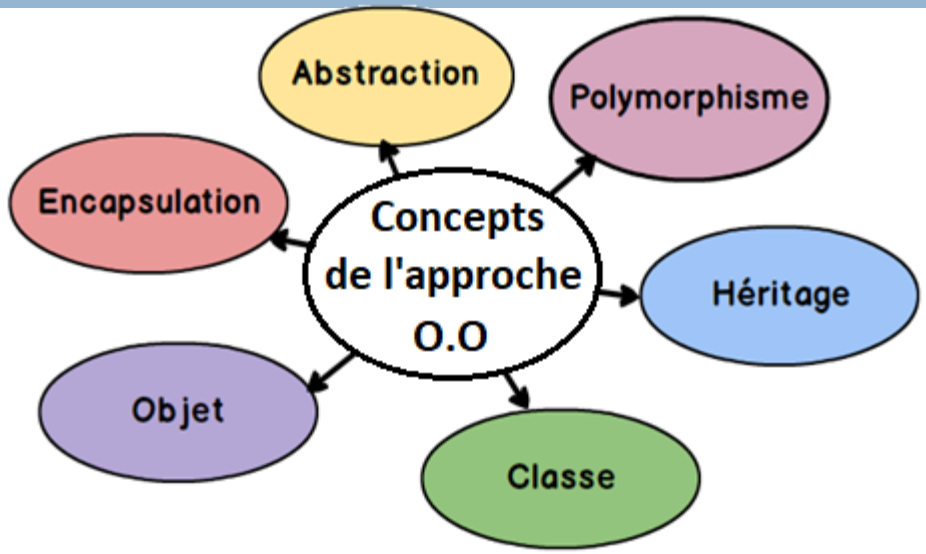
L'approche OO guide la conception par un ensemble de concepts

Concepts de la modélisation OO avec UML

Avantages …

- Grâce à ses concepts,





➔ l'**approche OO** reflète plus finement les **objets** du monde réel :

- Des **langages de modélisation OO** : **UML**.
- Des **langages de développement OO** : Java, Python, C++, ...
- Un code facile à maintenir avec possibilité de réutilisation.
- Un prototypage facile, ….

Concepts de la modélisation OO avec UML

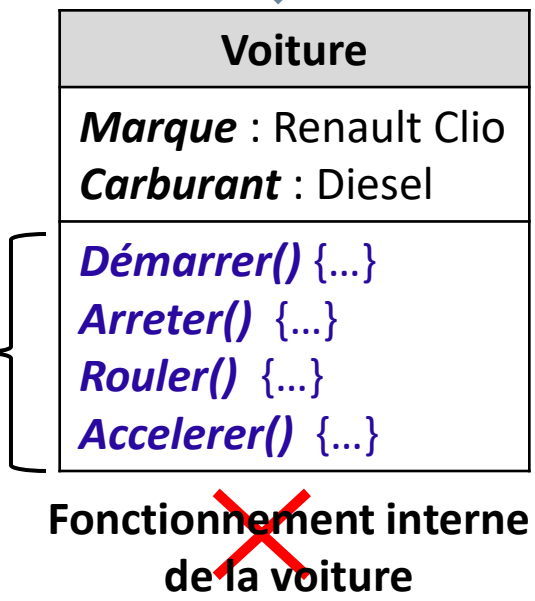
Abstraction …

- L'approche **OO** est basée sur le principe d'**abstraction de données**.
- L'**abstraction** est le processus qui consiste à représenter des **objets** du **monde réel** dans le **monde du programme**.



- Le **principe** de **l'approche OO** consiste à :

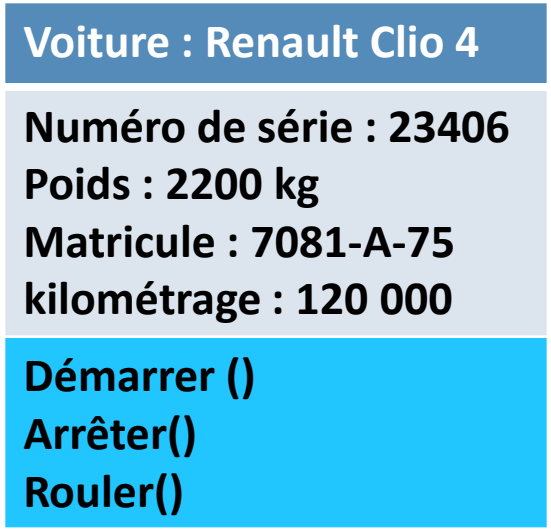
1) identifier les **objets** à manipuler.
2) déterminer les **traitements** à faire sur ces objets.
3) ne pas s'intéresser au **fonctionnement interne** des objets.



Concepts de la modélisation OO avec UML

Notion d'objet

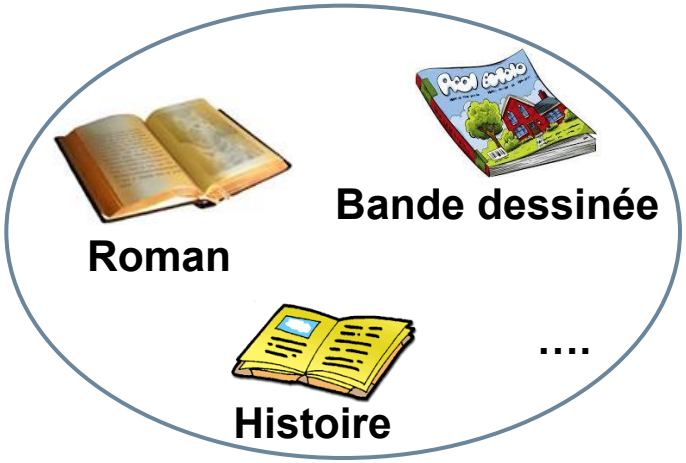
- Un **objet** se définit comme étant une **entité informatique** ayant :
  - une partie **statique** : *données=caractéristiques=attributs*
  - une partie **dynamique** : *méthodes=opérations=traitements*
- C'est une **abstraction** d'une entité du monde réel.
- **Caractérisation d'un objet :**
  - **Identité**
    - permet de le distinguer des autres objets
  - **Attributs**
    - données caractérisant l'objet
  - **Méthodes**
    - actions que l'objet peut réaliser



Concepts de la modélisation OO avec UML

Notion de classe

- Dans une application, les **objets** sont souvent **très nombreux**.
- **Exemple** : Dans une application de gestion d'une bibliothèque, on peut trouver plusieurs **objets** de **type livre** : ***Roman***, ***Histoire***, …



➔**On peut dégager un type d'objet** ➔



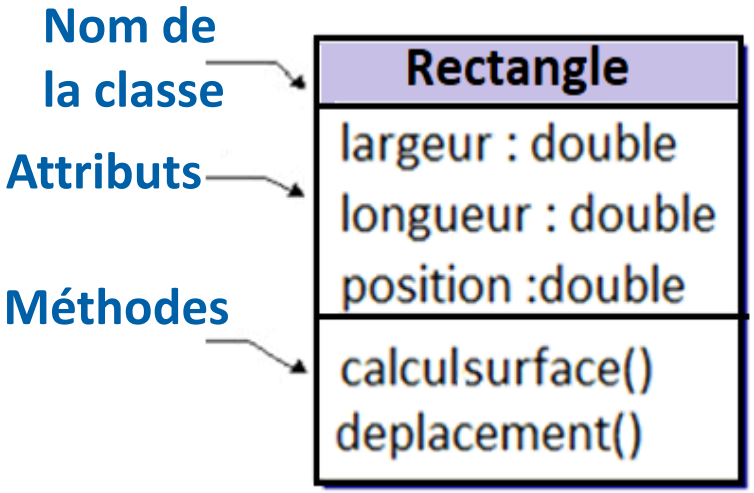
Livre

- La notion de **classe** correspond à cette notion de **type d'objet.**
- Un **objet** est une **variable** (instance) dont le **type** est celui de la **classe** dont il est issu.

Concepts de la modélisation OO avec UML

Notion de classe

- Une **classe** est une **entité** représentant la **structure** d'un **ensemble** d'**objets** ayant les mêmes **données** (attributs) et les mêmes **comportements** (méthodes).



- La classe contient des membres :
  - des **attributs** : les **valeurs** représentent l'**état de l'objet**.
  - des **méthodes** : définissent les **comportements** de l'**objet**.
- ➔ **Classe = Modèle d'objets.** La classe peut générer des **instances**.

Concepts de la modélisation OO avec UML

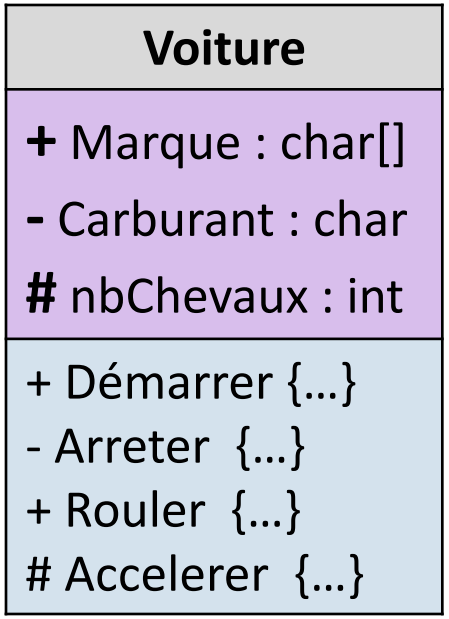
Encapsulation

- L'**encapsulation interdise** l'**accès direct** aux membres de l'**objet** en définissant des **modes de visibilité** (droits d'accès aux membres).
- Objectif : garantir l'**intégrité** des données de l'**objet**.
- Elle précise 3 modes de **visibilité** des **membres** d'un objet :

➲ **Publique (+)** : les membres sont **accessibles** par l'**objet lui même** ou par **n'importe quel autre objet**.

➲ **Privée (-)** : les membres sont **inaccessibles** à partir d'autres objets : seules les méthodes de l'**objet lui même** pourront y accéder.

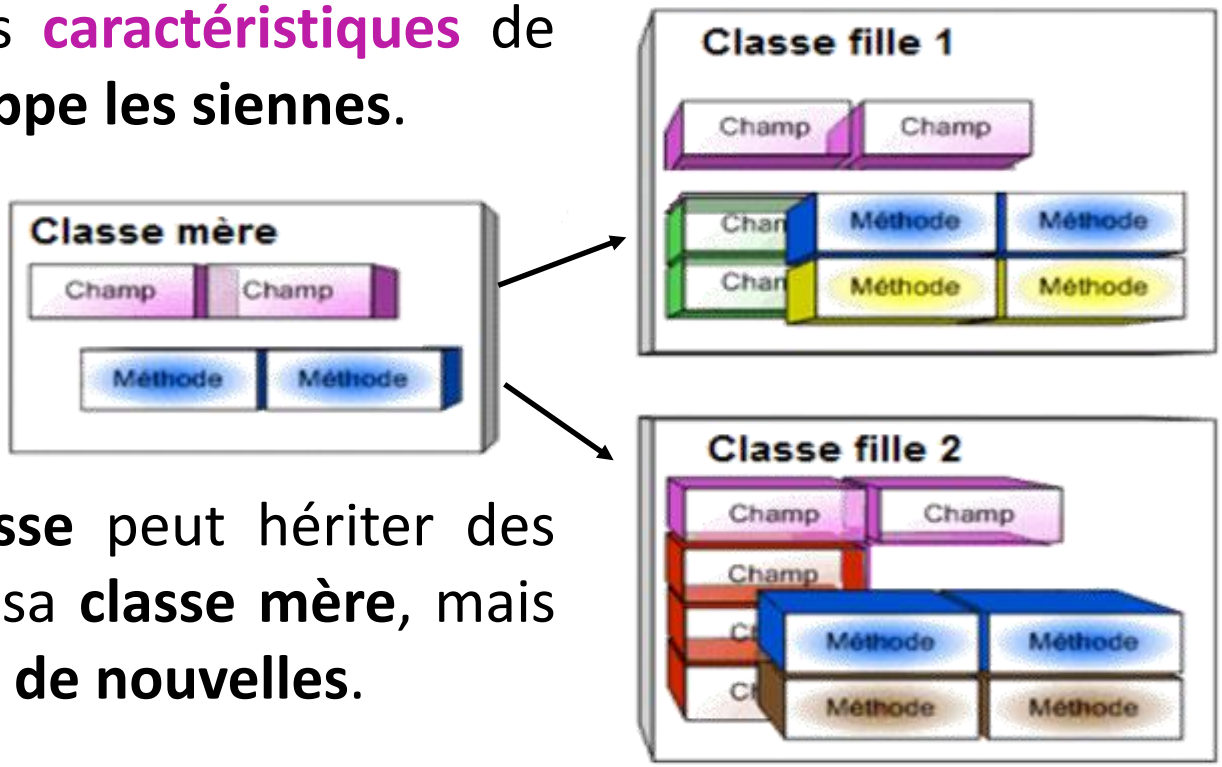
➲ **Protégée (#)** : les membres sont **accessibles** par les fonctions membres de la **classe elle-même** et par les **classes dérivées** (sous-classes).



Concepts de la modélisation OO avec UML

Héritage

- Un **enfant hérite** des **caractéristiques** de ses **parents** et **développe les siennes**.



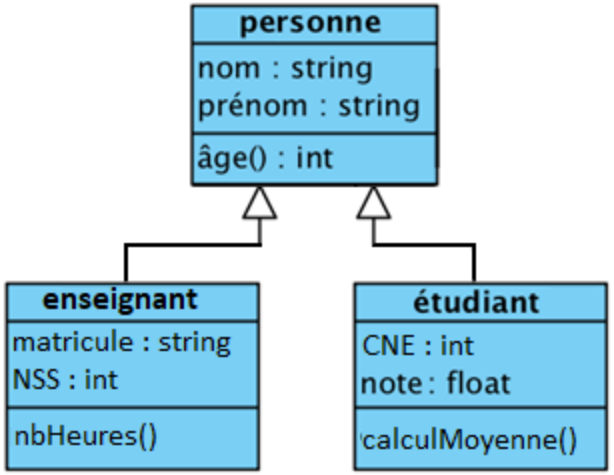
➔ De même, une **classe** peut hériter des **caractéristiques** de sa **classe mère**, mais aussi en **développer de nouvelles**.

- Une **classe B hérite** d'une **classe A**, ssi :
  - **B** possède les membres de **A**, **+** d'autres **membres** qui lui sont propres.
  - **B** redéfinie des **méthodes** déjà définies dans **A**.

Concepts de la modélisation OO avec UML

Héritage

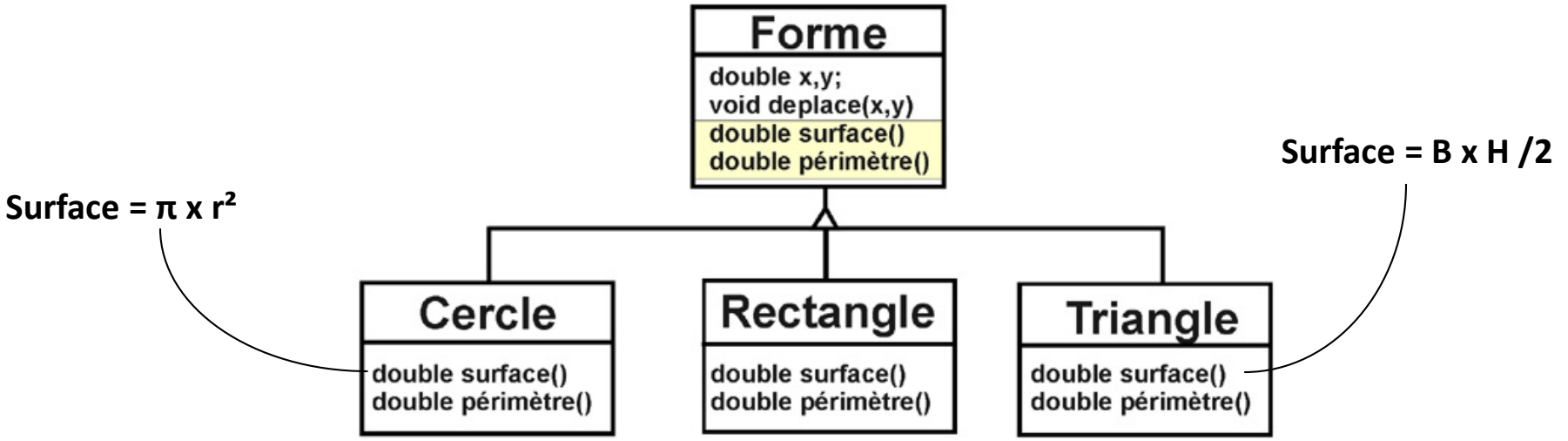
- L'**héritage** est une relation de **spécialisation/généralisation** entre deux classes.
- On dit que :
  - la **classe mère** **généralise** la **classe fille**.
  - la **classe fille** **spécialise** la **classe mère**.



- L'héritage peut être :
  - **simple :** la classe fille hérite d'une seule classe mère.
  - **multiple :** la classe fille hérite de plusieurs classes mères.

Concepts de la modélisation OO avec UML
Polymorphisme

- C'est un mécanisme permettant à une **classe fille** de **redéfinir** une **méthode** dont elle a **hérité** tout en gardant la **même signature**.
- **Exemple** : La méthode **surface()** présente dans la classe mère **Forme** est redéfinie dans ses classes filles **Cercle**, **Rectangle** et **Triangle** :



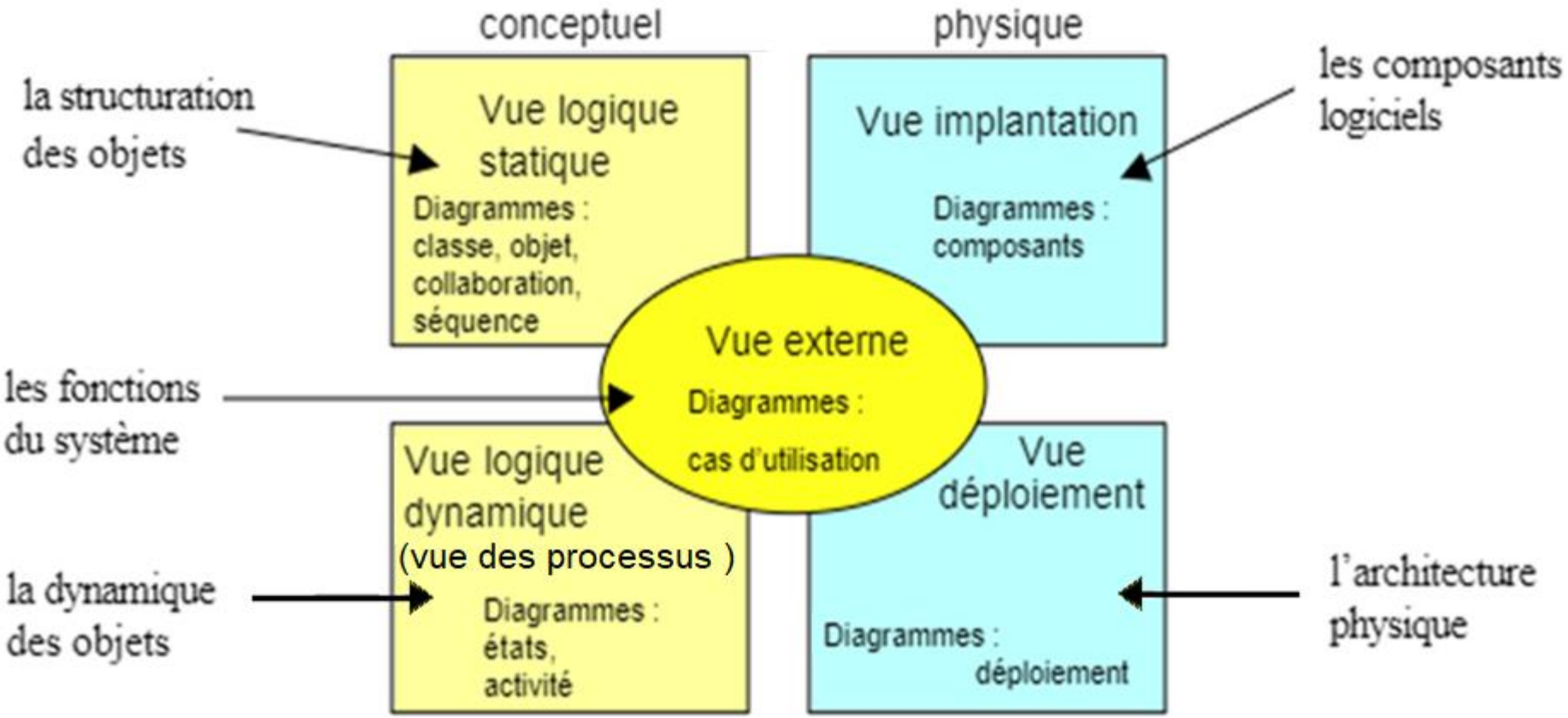
L'appel de la méthode **surface()** produira un **résultat différent** selon la sous classe.

Vues et diagrammes UML

Vues et diagrammes UML

Vues UML

- UML **propose** **5 vues** (sous ensembles du modèle) et **14 diagrammes** **:**



Vues et diagrammes UML

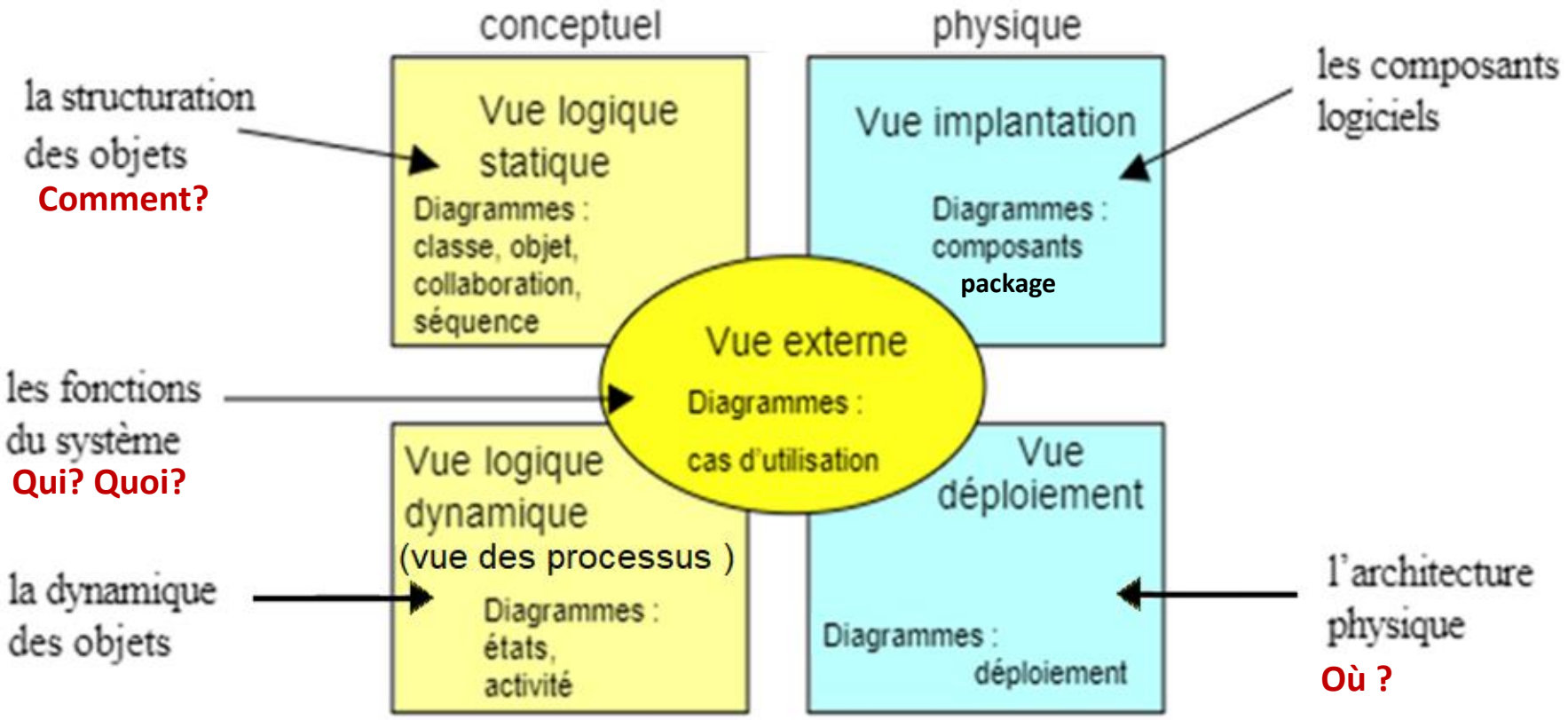
Vues UML

Vue externe (vue des cas d'utilisation)	Description (**diagrammes**) du système vu par les **acteurs** du système en représentant les **besoins** attendus par chaque acteur (c'est le **qui** et le **quoi**).
Vue logique statique	Description de l'architecture du système vu de **l'intérieur** : expliquer **comment** peuvent être satisfaits les besoins des acteurs en termes d'**objets** (**comment**).
Vue implémentation	Description de l'**organisation** des **modules** (exécutable, codes source, paquetages, etc.) dans l'environnement de développement.
Vue logique dynamique (des processus)	Vue **temporelle** et **technique** qui se rapporte aux **interactions** entre **objets** actifs : tâches concurrentes, synchronisation, etc.
Vue déploiement	Description de l'architecture **physique** (**matérielle**) de chaque élément du système (c'est le **OÙ**).

Vues et diagrammes UML

Diagrammes UML

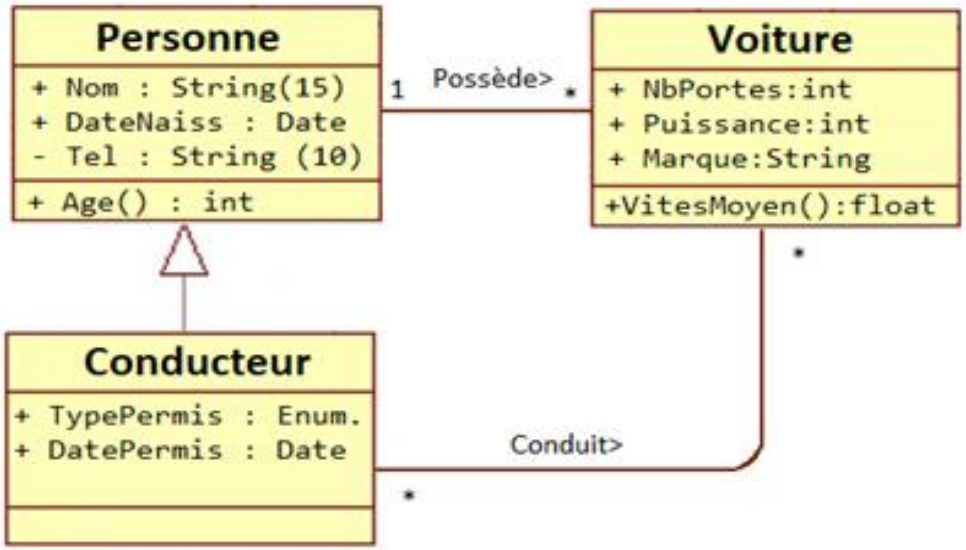
- Un **diagramme UML** est une **représentation graphique**, qui s'intéresse à un **aspect** précis (statique ou dynamique) du **modèle**.



Vues et diagrammes UML

Diagrammes UML

- Chaque **type** de **diagramme UML** possède une **structure.**
- **Exemple :** Exemple simplifié de diagramme de classes.



- Une fois combinés, les **différents types de diagrammes UML** offrent une **vue complète** des aspects **statiques** et **dynamiques** d'un **système**.

Vues et diagrammes UML

Diagrammes UML

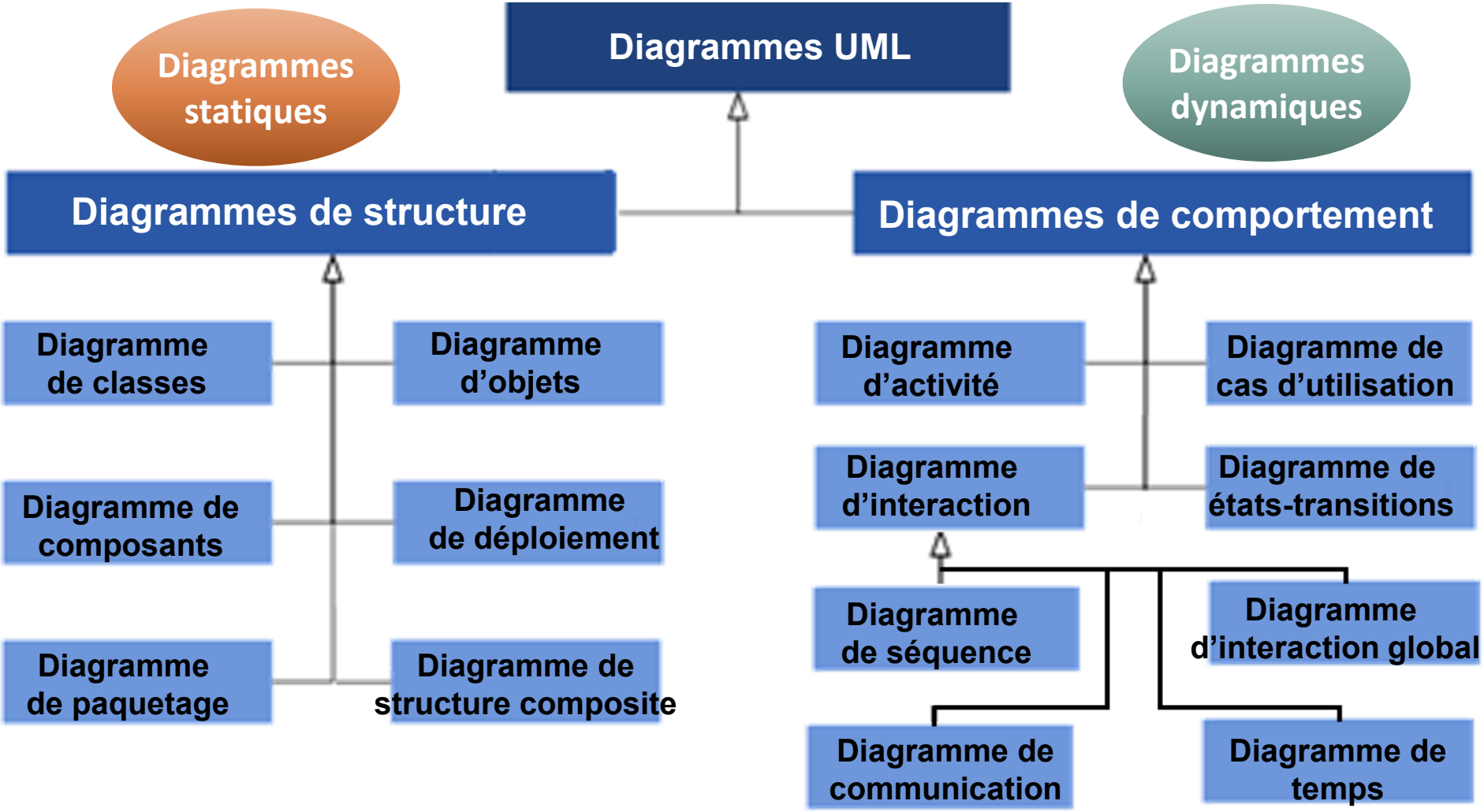
- UML 2 propose **14 types** de **diagrammes** représentant autant de **vues** pour représenter des concepts particuliers du système.

- Ils se répartissent en deux grands groupes :
  - **7 diagrammes structurels** (statiques) : on ne tient pas compte du facteur **temporel** dans le comportement du système.
  - **7 diagrammes comportementaux** (dynamiques).

- Ces **diagrammes** sont dépendants et se complètent de façon à permettre la modélisation d'un **projet informatique** tout au long de son **cycle de vie**.

Vues et diagrammes UML

Diagrammes UML



Vues et diagrammes UML

Diagrammes UML

- ***Remarques*** :
  - Les **diagrammes UML** **ne sont pas nécessairement tous produits** lors d'une modélisation.
  - Les diagrammes les plus utilisés sont :
    - Le diagramme des **cas d'utilisation** ;
    - Le diagramme de **séquence** ;
    - Le diagramme de **classes**.

Vues et diagrammes UML

Diagrammes UML

- **Diagrammes structurels (Statiques)**

Diagramme de classes	Décrit la conception logique et physique d'un système un visuel des différentes **classes** et de leur interdépendance.
Diagramme d'objets	Décrit les **instances** des éléments structurels et leurs liens à l'exécution.
Diagramme de packages	Décrit les dépendances entre les **paquetages** qui composent un modèle. L'objectif principal est de montrer la relation entre les différents grands composants qui forment un système complexe.
Diagramme de composants	Offre une vue plus simplifiée d'un système complexe en le désossant en plus petits composants. Les connecteurs définissent la relation/dépendances entre les composants.
Diagramme de déploiement	Décrit les composants matériels (nœuds) et logiciels (artefacts) ainsi que leurs relations. Il apporte une représentation visuelle de l'endroit exact où chaque composant logiciel est déployé.

Vues et diagrammes UML

Diagrammes UML

- **Diagrammes comportementaux (dynamiques)**

Diagramme de cas d'utilisation	Décrit les **interactions fonctionnelles** entre les **acteurs** et le **système** (montre ce que fait un système).
Diagramme global d'interactions	Fusionne les **diagrammes d'activité** et de **séquence** pour combiner des fragments d'interaction avec des décisions et des flots.
Diagramme de séquence	Décrit la séquence verticale des **messages** passés entre **objets** au sein d'une interaction.
Diagramme de communication	Décrit la **communication** entre **objets** dans le plan au sein d'une **interaction**.
Diagramme d'activité	Décrit l'**enchaînement** des **actions** et **décisions** au sein d'une **activité**.
Diagramme d'états	Décrit les différents **états** et **transitions** possibles des **objets** d'une classe.